CENTRO PAULA SOUZA

ETEC UIRAPURU

Desenvolvimento de Sistemas

Isaias Rocha De Jesus

Exercícios – BD II

São Paulo

2026

Sumário

Modelo Conceitual -Modelo Lógico – 18.03.2026 3

Exercícios: 3

SELECT - 04.04.2026 8

Continuação - 22.04.2026 14

Operadores Lógicos – 29.04.2026 15

Operadores BETWEEN e NOT BETWEEN 18

Explicação: 18

Operadores LIKE e NOT LIKE – 06.05.2026 20

Explicação: 20

Exemplos: 20

Operadores IN e NOT IN 23

Explicação 23

Operadores IS NULL e IS NOT NULL 24

Explicação: 24

Ordenando os Dados Selecionados 25

Explicação**:** 25

Realizando Cálculos sobre a Informação Selecionada 29

Totalizando Colunas (SUM) 31

Calculando Médias (AVG) 31

Contando as Linhas (COUNT) 32

A Palavra-Chave DISTINCT 33

Agrupando a Informação Selecionada (GROUP BY) 34

A Cláusula HAVING 35

Modelo Conceitual -Modelo Lógico – 18.03.2026

Exercícios:

Construa os modelos lógicos nas formas Textual e Gráfica para os modelos conceituais a seguir:

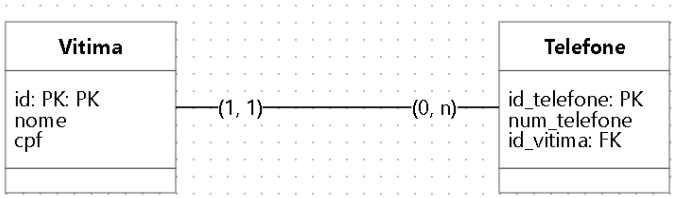
Exercício 01 – Vitima:

Textual:

VITIMA (vitima_id :PK, nome, cpf)

VITIMA_TELEFONE (id_telefone: PK, num_telefone, id_vitima: FK)

Lógico:

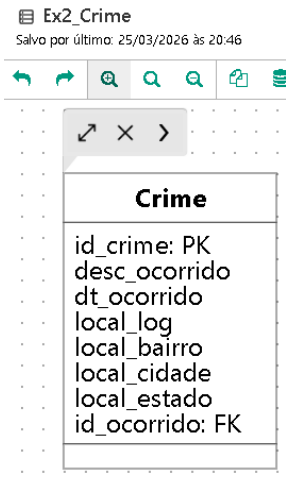


Exercício 02 – Crime:

Textual:

CRIME (crime_id PK, desc_ocorrido, dt_ocorrido, id_ocorrido: PK, local_logradouro, local_bairro, local_cidade, local_stado);

Lógico:



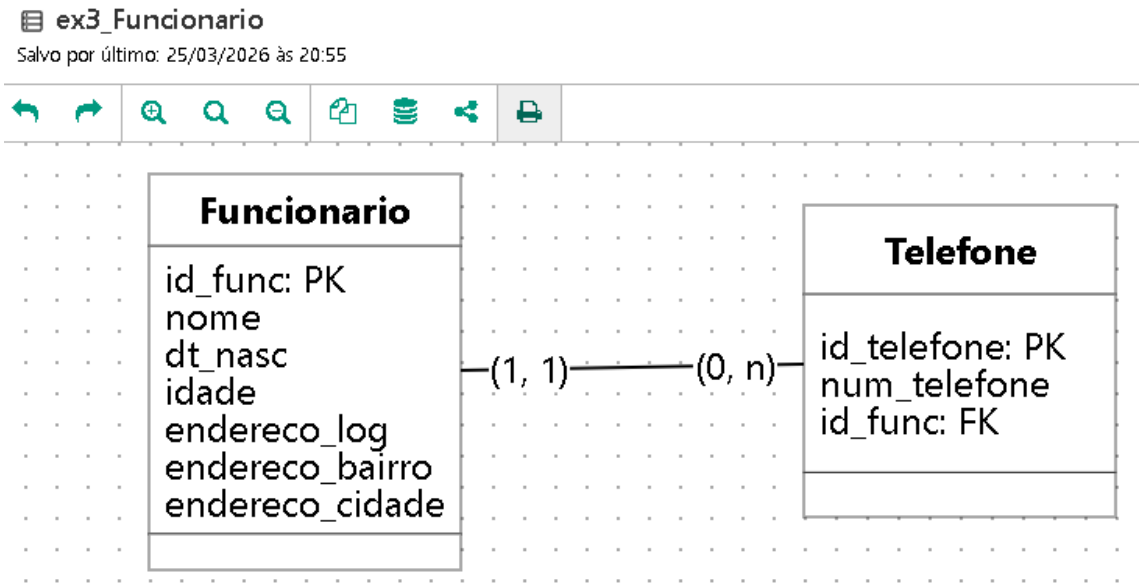
Exercício 03 – Funcionário:

Textual:

FUNCIONARIO (Id_func FK, nome, dt_nasc, idade, endereco_log, endereco_bairro, endereco_cidade);

TELEFONE (id_telefone: PK, num_telefone, (id_func: FK, telefone);

Lógico:



Exercício 04 – Aluno:

Textual:

ALUNO (id_aluno: PK, nome)

ALUNO_NECESSIDADE_ESPECIAL (id_necessidade: PK, desc_necessidade, id_aluno: FK)

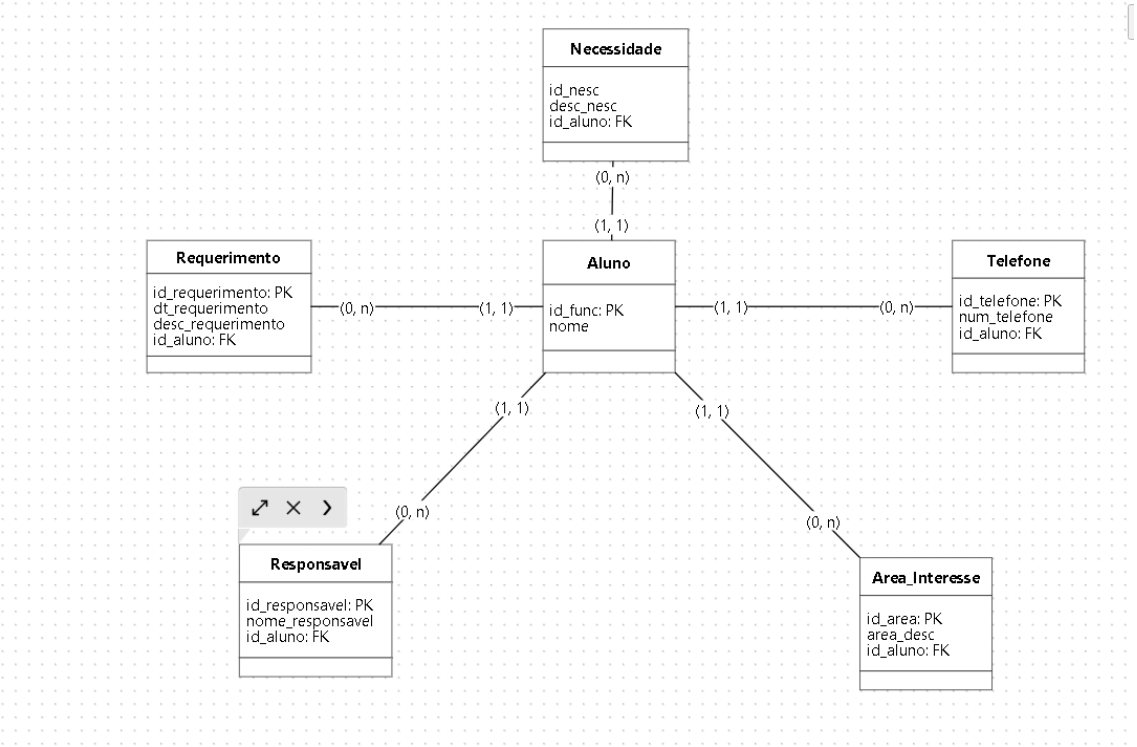
ALUNO_TELEFONE (id_telefone: PK, num_telefone, id_aluno: FK)

ALUNO_AREA_INTERESSE (id_area: PK, area_interesse, id_aluno: FK)

ALUNO_RESPONSAVEL (id_responsavel: PK, nome_responsavel, id_aluno: FK)

REQUERIMENTO (id_requerimento: PK, dt_requerimento, desc_requerimento id_aluno: FK)

Lógico:



SELECT - 04.04.2026

Problema 1:

Liste todos os produtos com as respectivas descrições, unidades e valores unitários.

Código: SELECT desc_prod, unid_prod, val_unit FROM `PRODUTO`;

Mostrando registros 0 - 4 (5 no total, Consulta levou 0.0002 segundos.)

```
SELECT desc_prod, unid_prod, val_unit FROM `produto`;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabe

Opções extras

←T→	desc_prod	unid_prod	val_unit
Editar Copiar Remover	Chapa de Aco	kg	2.50
Editar Copiar Remover	Cimento	kg	4.50
Editar Copiar Remover	parafuso 3.0X10.5 mm	kg	2.00
Editar Copiar Remover	Fio plastico	m	2.00
Editar Copiar Remover	Solvente PRW	l	5.00

Problema 2:

Liste os nomes dos clientes, as cidades e os estados onde eles estão localizados.

Código: SELECT nome_cli, cidade, uf FROM `cliente`;

```
SELECT nome_cli, cidade, uf FROM `cliente`;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta

Opções extras

Editar	Copiar	Remover	Supermercado Carrefour	rio de janeiro	rj
Editar	Copiar	Remover	Supermercado Baratao	rio de janeiro	rj
Editar	Copiar	Remover	Supermercado Arariboia	niteroi	rj

Servidor: 127.0.0.1 » Banco de dados: modelo_ds2 » Tabela: cliente

Visualizar | Estrutura | SQL | Procurar | Inserir | Ex

			nome_cli	cidade	uf
Editar	Copiar	Remover	Lojas Pague Pouco	sao paulo	sp
Editar	Copiar	Remover	Ford Caminhoes	sao paulo	sp
Editar	Copiar	Remover	Célula Celulose	guaiba	rs
Editar	Copiar	Remover	Elevadores RioSul	guaiba	rs

Problema 3:

Listar o conteúdo de todas as tabelas de base de dados dos exemplos.

3.1. Cliente:

Código:

```
SELECT * FROM `cliente`;
```

```
SELECT * FROM cliente;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Ordenar pela chave: Nen

Opções extras

←T→			cod_cli	nome_cli	endereco	cidade	cep	uf
Editar	Copiar	Remover	1000	Supermercado Carrefour	Av. das Americas	rio de janeiro	20000001	rj
Editar	Copiar	Remover	2000	Supermercado Baratao	Rua 7 de setembro	rio de janeiro	20000002	rj
Editar	Copiar	Remover	3000	Supermercado Arariboia	Rua Itaoca	niteroi	20000003	rj
Editar	Copiar	Remover	4000	Mercado São João	Cidade Univers.	niteroi	20000004	rj
Editar	Copiar	Remover	5000	CSN	Rua das Nações	volta redonda	20000005	rj
Editar	Copiar	Remover	6000	Pegeout	Rodovia Pres. Dutra	resende	20000006	rj
Editar	Copiar	Remover	7000	Lojas Pague Pouco	Rua Tuiuti	sao paulo	11000001	sp
Editar	Copiar	Remover	8000	Ford Caminhoes	Rua Henry Ford	sao paulo	11000002	sp
Editar	Copiar	Remover	9000	Célula Celulose	Rua Gen. Arouca	guaiba	30000001	rs
Editar	Copiar	Remover	10000	Elevadores RioSul	Rua Planejada	guaiba	30000001	rs

3.2. Item_Pedido:

Código:

```
SELECT * FROM `item_pedido`;
```

no_ped	cd_prod	qtd_ped
1111	100	100
1111	200	100
1111	300	200
1112	400	100
1112	500	1000
1113	100	300
2111	100	500
2111	500	400
2112	200	100
2112	300	200
2113	500	500
3111	400	300
3112	100	400
3112	200	600
4111	300	700
4112	500	1000
4112	200	500
5111	200	100
5111	300	500
6111	400	100
6112	300	400
6112	400	200
7111	100	500

3.3. Pedido:

Código:

```
SELECT * FROM `pedido`;
```

	num_ped	prazo_entr	cd_cli	cd_vend
over	1111	10	1000	11
over	1112	5	1000	11
over	1113	30	1000	15
over	2111	15	3000	14
over	2112	18	3000	15
over	2113	3	3000	12
over	3111	13	4000	12
over	3112	7	4000	11
over	4111	7	6000	11
over	4112	7	6000	14
over	5111	10	8000	14
over	6111	30	9000	14
over	6112	60	9000	12
over	7111	20	10000	15

3.4. Produto:

Código:

```
SELECT * FROM `produto`;
```

			cod_prod	unid_prod	desc_prod	val_unit
Editar	Copiar	Remover	100	kg	Chapa de Aco	2.50
Editar	Copiar	Remover	200	kg	Cimento	4.50
Editar	Copiar	Remover	300	kg	parafuso 3.0X10.5 mm	2.00
Editar	Copiar	Remover	400	m	Fio plastico	2.00
Editar	Copiar	Remover	500	l	Solvente PRW	5.00

3.5. Vendedor:

Código:

```
SELECT * FROM `vendedor`;
```

```
SELECT * FROM `vendedor`
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela

Opções extras

			cod_vend	nome_vend	sal_fixo	faixa_comiss
Editar	Copiar	Remover	11	Paulo Alberto	1500.00	b
Editar	Copiar	Remover	12	Ana Cristina	2100.00	a
Editar	Copiar	Remover	13	Cassia Andrade	900.00	c
Editar	Copiar	Remover	14	João Roberto	2500.00	a
Editar	Copiar	Remover	15	Maria Paula	900.00	c

Problema 4:

Listar o número de pedido, o código do produto e a quantidade pedida dos itens de um pedido, onde a quantidade pedida seja igual a 500.

Código:

```
SELECT no_ped, cd_prod, qtd_ped

FROM item_pedido

WHERE qtd_ped = 500;
```

```
select no_ped, cd_prod, qtd_ped FROM item_pedido WHERE qtd_ped = 500;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualiza

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar ne

Opções extras

no_ped	cd_prod	qtd_ped
2111	100	500
2113	500	500
4112	200	500
5111	300	500
7111	100	500

Problema 5:

SELECT no_ped, cd_prod, qtd_ped

FROM `item_pedido`

WHERE qtd_ped = 500;

```
SELECT no_ped, cd_prod, qtd_ped FROM `item_pedido` WHERE qtd_ped = 500;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta

Opções extras

no_ped	cd_prod	qtd_ped
2111	100	500
2113	500	500
4112	200	500
5111	300	500
7111	100	500

Continuação - 22.04.2026

Problema 6:

Quais são os clientes na cidade de Niterói?

Código:

SELECT nome_cli, cidade

FROM `cliente`

WHERE cidade = ‘niteroi’ ;

Mostrando registros 0 - 1 (2 no total, Consulta levou 0.0004 segundos.)

```
SELECT nome_cli, cidade FROM `cliente` WHERE cidade = 'niteroi';
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atu

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procura

Opções extras

←T→			nome_cli	cidade
Editar	Copiar	Remover	Supermercado Arariboia	niteroi
Editar	Copiar	Remover	Mercado São João	niteroi

Operadores Lógicos – 29.04.2026

AND	Conjunção
OR	Disjunção
NOT	Negação

Problema 7:

Quais são os produtos que têm unidade igual a ‘kg’ e valor maior do que R$ 2,00?

Código:

```
SELECT unid_prod,val_unit

FROM `produto`

WHERE unid_prod  =  'kg' AND val_unit  >  2.00;
```

Mostrando registros 0 - 1 (2 no total, Consulta levou 0.0007 segundos.)

```
SELECT unid_prod,val_unit FROM `produto` WHERE unid_prod = 'kg' AND val_unit > 2.00;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Ordenar p

Opções extras

←T→	unid_prod	val_unit
Editar Copiar Remover	kg	2.50
Editar Copiar Remover	kg	4.50

Problema 8:

Problema 9:

Liste todos os clientes localizados na cidade de São Paulo ou que tenham CEP entre 20000005 e 200000010.

Código:

SELECT nome_cli,cidade,cep

FROM `cliente`

WHERE cidade = 'são paulo' AND cep >= 20000005 <=20000010;

Mostrando registros 0 - 1 (2 no total, Consulta levou 0.0004 segundos.)

```
SELECT nome_cli,cidade,cep FROM `cliente` WHERE cidade = 'são paulo' AND cep >= 20000005 <=20000010;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Ordenar pela chave:

Opções extras

			nome_cli	cidade	cep
Editar	Copiar	Remover	Lojas Pague Pouco	sao paulo	11000001
Editar	Copiar	Remover	Ford Caminhoes	sao paulo	11000002

<u>Observação:</u>

A prioridade de operador ***AND*** é maior do que a prioridade do operador ***OR***; logo, neste exemplo, a utilização dos parênteses é opcional.

Problema 10:

Mostrar todos os pedidos que não tenham prazo de entrega superior a 15 dias.

Código:

```
SELECT  num_ped, prazo_entr

FROM `pedido`

WHERE prazo_entr  < 15
```

```
SELECT num_ped, prazo_entr FROM `pedido` WHERE prazo_entr < 15;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualiz

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar n

Opções extras

←T→				num_ped	prazo_entr
	Editar	Copiar	Remover	1111	10
	Editar	Copiar	Remover	1112	5
	Editar	Copiar	Remover	2113	3
	Editar	Copiar	Remover	3111	13
	Editar	Copiar	Remover	3112	7
	Editar	Copiar	Remover	4111	7
	Editar	Copiar	Remover	4112	7
	Editar	Copiar	Remover	5111	10

Operadores BETWEEN e NOT BETWEEN

Explicação:

WHERE <mome_coluna> BETWEEN <valor 1> AND <valor 2>

WHERE <mome_coluna> NOT BETWEEN <valor 1> AND <valor 2>

Este operador possibilita a seleção de uma faixa de valores sem a necessidade do uso dos operadores >=, <= e AND.

<valor 1> e <valor 2> têm que ter o mesmo tipo de dado que <nome da coluna>

Problema 11:

Liste o código 8e a descrição dos produtos que tenham o valor unitário na faixa de R$ 0,10 a R$ 3,00.

Código:

SELECT cod_prod,desc_prod,val_unit

FROM `produto`

WHERE val_unit BETWEEN 0.10 AND 3.00;

```
SELECT cod_prod,desc_prod,val_unit FROM `produto` WHERE val_unit BETWEEN 0.10 AND 3.00;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Orde

Opções extras

←T→			cod_prod	desc_prod	val_unit
Editar	Copiar	Remover	100	Chapa de Aco	2.50
Editar	Copiar	Remover	300	parafuso 3.0X10.5 mm	2.00
Editar	Copiar	Remover	400	Fio plastico	2.00

Operadores LIKE e NOT LIKE – 06.05.2026

- WHERE <nome_coluna> **LIKE** <valor>

- WHERE <nome_coluna> **NOT LIKE** <valor>

Explicação:

Aplicáveis apenas a colunas dos tipos CHAR e VARCHAR.

Funcionam de modo análogo aos operadores = e <>, porém o poder dos operadores LIKE e NOT LIKE está na utilização dos símbolos % e que podem fazer o papel de "coringa":

Símbolo	Descrição
%	substitui uma palavra
_ (underline)	substitui um caracter qualquer

Exemplos:

'apis%' se aplicaria às seguintes cadeias de caracteres:

'lapis preto'

‘lapis cera'

'lapis borracha'

*'broca n_*' se aplicaria às seguintes cadeias de caracteres:

'broca n1'

'broca n9'

'broca n3'

Problema 12:

Listar todos os produtos que tenham a sua unidade começando por k (lembre-se de que a coluna unid_prod foi definida como char(03)).

Código:

```
SELECT *
FROM `produto`
WHERE unid_prod
LIKE 'k%';
```

```
SELECT * FROM `produto` WHERE unid_prod LIKE 'k_';
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Ordena

Opções extras

←T→	cod_prod	unid_prod	desc_prod	val_unit
Editar Copiar Remover	100	kg	Chapa de Aco	2.50
Editar Copiar Remover	200	kg	Cimento	4.50
Editar Copiar Remover	300	kg	parafuso 3.0X10.5 mm	2.00

Problema 13:

Listar todos os vendedores cujos os nomes não comecem por 'A'.

Código: SELECT *

FROM `cliente`

WHERE nome_cli

NOT LIKE 'A%';

Mostrando registros 0 - 3 (4 no total, Consulta levou 0.0003 segundos.)

```
SELECT nome_vend FROM `vendedor` WHERE nome_vend NOT LIKE "A%";
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualiz

Mostrar tudo | Número de linhas: 25

Opções extras

←T→			nome_vend
Editar	Copiar	Remover	Paulo Alberto
Editar	Copiar	Remover	Cassia Andrade
Editar	Copiar	Remover	João Roberto
Editar	Copiar	Remover	Maria Paula

Operadores IN e NOT IN

WHERE <nome_coluna> **IN** <lista_de_valores>

WHERE <nome_coluna> **NOT IN** (<lista_de_valores>)

Explicação

Seleciona as linhas cujo o valor da coluna <nome_coluna> pertença ao conjunto <lista_de_valores>.

Problema 14:

Listar todos os vendedores cujas as faixas de comissão sejam 'a' ou 'b'.

Código:

SELECT *

FROM `vendedor`

WHERE faixa_comiss

IN ('a' , 'b');

```
SELECT nome_vend, faixa_comiss FROM `vendedor` WHERE faixa_comiss in ('a','b');
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25

Opções extras

←T→			nome_vend	faixa_comiss
Editar	Copiar	Remover	Paulo Alberto	b
Editar	Copiar	Remover	Ana Cristina	a
Editar	Copiar	Remover	João Roberto	a

Operadores IS NULL e IS NOT NULL

WHERE <nome_coluna> **IS NULL**

WHERE <nome_coluna> **IS NOT NULL**

Explicação:

A utilização do valor nulo (NULL) é problemática, pois as diversas implementações da linguagem SQL podem adotar qualquer representação para o valor nulo.

Problema 15:

Mostrar os clientes que não tenham endereço cadastrado.

Código:

SELECT nome_cli, endereco

FROM `cliente`

WHERE endereco IS NULL;

```
SELECT * FROM `cliente` WHERE endereco IS null;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [C

cod_cli	nome_cli	endereco	cidade	cep	uf

Ordenando os Dados Selecionados

SELECT <lista_de_colunas>

FROM <nome_tabela>

WHERE <condição_de_seleção>

ORDER BY {<nome_coluna> | <num_col> [ASC | DESC]}

Explicação:

Onde <nome_coluna> se refere à coluna segundo a qual as linhas serão ordenadas, e <num_col> se refere à posição relativa da coluna na <lista_de_colunas> projetadas, contada da esquerda para a direita, e não à posição na tabela original.

As cláusulas **ASC** e **DESC** denotam ordenação ascendente e descendente respectivamente. A forma ascendente de ordenação é assumida caso nenhuma opção seja informada explicitamente.

Problema 16:

Mostrar em ordem alfabética a lista de vendedores e seus respectivos salários fixos.

Código:

```
SELECT nome_vend, sal_fixo
FROM vendedor
WHERE sal_fixo
 ORDER BY nome_vend ASC;
```

```
SELECT nome_vend, sal_fixo FROM vendedor WHERE sal_fixo ORDER BY nome_vend ASC;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela

Opções extras

←T→			nome_vend 1	sal_fixo
Editar	Copiar	Remover	Ana Cristina	2100.00
Editar	Copiar	Remover	Cassia Andrade	900.00
Editar	Copiar	Remover	João Roberto	2500.00
Editar	Copiar	Remover	Maria Paula	900.00
Editar	Copiar	Remover	Paulo Alberto	1500.00

Problema 17:

Listar os nomes, as cidades e os estados de todos os clientes, ordenados por estado e cidade de forma descendente.

Código:

```
SELECT *

FROM `cliente`

ORDER BY `endereco`,`uf`

DESC;
```

```
SELECT * FROM `cliente` ORDER BY `endereco`,`uf` DESC;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Ordenar pela chave: Nenhum

Opções extras

←T→			cod_cli	nome_cli	endereco 1	cidade	cep	uf 2
Editar	Copiar	Remover	1000	Supermercado Carrefour	Av. das Americas	rio de janeiro	20000001	rj
Editar	Copiar	Remover	4000	Mercado São João	Cidade Univers.	niteroi	20000004	rj
Editar	Copiar	Remover	6000	Pegeout	Rodovia Pres. Dutra	resende	20000006	rj
Editar	Copiar	Remover	2000	Supermercado Baratao	Rua 7 de setembro	rio de janeiro	20000002	rj
Editar	Copiar	Remover	5000	CSN	Rua das Nações	volta redonda	20000005	rj
Editar	Copiar	Remover	9000	Célula Celulose	Rua Gen. Arouca	guaiba	30000001	rs
Editar	Copiar	Remover	8000	Ford Caminhoes	Rua Henry Ford	sao paulo	11000002	sp
Editar	Copiar	Remover	3000	Supermercado Arariboia	Rua Itaoca	niteroi	20000003	rj
Editar	Copiar	Remover	10000	Elevadores RioSul	Rua Planejada	guaiba	30000001	rs
Editar	Copiar	Remover	7000	Lojas Pague Pouco	Rua Tuiuti	sao paulo	11000001	sp

Problema 18:

Mostrar a descrição e o valor unitário de todos os produtos que tenham unidade 'kg' em ordem ascendente de valor unitário.

Código:

```
SELECT desc_prod, val_unit

FROM produto

WHERE unid_prod = 'kg'

ORDER BY val_unit ASC;
```

Mostrando registros 0 - 2 (3 no total, Consulta levou 0.0004 segundos.) [val_unit: **2.00... - 4.50...**]

```
SELECT desc_prod, val_unit FROM produto WHERE unid_prod = 'kg' ORDER BY val_unit ASC;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Or

Opções extras

	desc_prod	val_unit 1
Editar Copiar Remover	parafuso 3.0X10.5 mm	2.00
Editar Copiar Remover	Chapa de Aco	2.50
Editar Copiar Remover	Cimento	4.50

Realizando Cálculos sobre a Informação Selecionada

Podemos criar dinamicamente um campo que não pertença à tabela original através de operações executadas sobre os campos projetados.

Problema 19:

Exibir o novo salário fixo dos vendedores da faixa de comissão 'C', calculado com base no reajuste de 75% sobre o salário atual acrescido de R$ 120,00 de bonificação. Ordene a relação resultante pelo nome do vendedor

Código:

```
SELECT nome_vend, (sal_fixo * 0.75) + 120.00
AS novo_salario
FROM vendedor
WHERE faixa_comiss = 'c'
ORDER BY nome_vend ASC;
```

Problema 20:

Máximos (MAX) e Mínimos (MIN)

Mostrar o menor e o maior salário entre os vendedores.

Código:

SELECT MIN(sal_fixo), Max(sal_fixo)

from vendedor;

Mostrando registros 0 - 0 (1 no total, Consulta levou 0.0003

```
SELECT MIN(sal_fixo), Max(sal_fixo) from vendedor;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar

Mostrar tudo | Número de linhas: 25 Filtra

Opções extras

MIN(sal_fixo)	Max(sal_fixo)
900.00	2500.00

Totalizando Colunas (SUM)

Problema 21:

Mostrar a quantidade total pedida para o produto cimento, de código 200

```
SELECT cd_prod, SUM(qtd_ped) AS quantidade_total FROM item_pedido WHERE cd_prod = 200 GROUP BY cd_prod;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela

Opções extras

cd_prod	quantidade_total
200	1400

Calculando Médias (AVG)

Problema 22:

Qual é a média dos salários fixos dos vendedores?

Código:

SELECT AVG (sal_fixo) A

S media_salarios

FROM vendedor;

```
SELECT AVG (sal_fixo) AS media_salarios FROM vendedor;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar códi

Mostrar tudo | Número de linhas: 25 Filtrar lin

Opções extras

media_salarios
1580.000000

Contando as Linhas (COUNT)

Problema 23:

Quantos vendedores ganham acima de R$ 2.000,00 de salário fixo?

Código:

SELECT COUNT(*) AS quantidade_vendedores

FROM vendedor

WHERE sal_fixo > 2000.00;

```
SELECT COUNT(*) AS quantidade_vendedores FROM vendedor WHERE sal_fixo > 2000.00;
```

☐ Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Opções extras

quantidade_vendedores
2

A Palavra-Chave DISTINCT

Várias linhas de uma tabela podem conter os mesmos valores para as suas colunas (duplicidade), com exceção da chave primária. Quando desejarmos eliminar a duplicidade, podemos inserir a palavra-chave DISTINCT após a palavra-chave SELECT.

Problema 24:

Em que cidades as nossas empresas possuem clientes?

Código:

SELECT DISTINCT cidade FROM cliente;

```
SELECT DISTINCT cidade FROM cliente;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [

Mostrar tudo | Número de linhas: 25

Opções extras

←T→			cidade
Editar	Copiar	Remover	rio de janeiro
Editar	Copiar	Remover	niteroi
Editar	Copiar	Remover	volta redonda
Editar	Copiar	Remover	resende
Editar	Copiar	Remover	sao paulo
Editar	Copiar	Remover	guaiba

Agrupando a Informação Selecionada (GROUP BY)

Existem ocasiões em que desejamos aplicar uma função de agregação não somente a um conjunto de tuplas, mas também organizar a informação em determinadas categorias. Isto é possível através do uso da cláusula GROUP BY.

Problema 25:

Listar o número de itens existente em cada pedido.

Código:

```
SELECT no_ped, COUNT(*)

AS numero_itens

FROM item_pedido

GROUP BY no_ped;
```

```
SELECT no_ped, COUNT(*) AS numero_itens FROM item_pedido GROUP BY no_ped;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta ta

Opções extras

no_ped	numero_itens
1111	3
1112	2
1113	1
2111	2
2112	2
2113	1
3111	1
3112	2
4111	1
4112	2
5111	2
6111	1
6112	2
7111	1

Inicialmente as linha são agrupadas através do(s) atributo(s) fornecido(s) na cláusula GROUP BY; neste caso, no_ped. Em um segundo passo, é aplicada a operação COUNT(*) para cada grupo de linhas que tenha o mesmo número de pedido. Após a operação de contagem de cada grupo, o resultado da consulta é apresentado.

Normalmente, a cláusula GROUP BY é utilizada em conjunto com as operações COUNT e AVG.

A Cláusula HAVING

Às vezes temos que definir condições e aplicá-las aos grupos ao invés de fazê-lo a cada linha separadamente. Por exemplo, suponha que desejemos listar todos os pedidos que possuam mais de um item. Esta condição não se aplica a uma única linha separadamente, mas a cada grupo definido pela cláusula GROUP BY. Para exprimir tal consulta, usamos a cláusula HAVING. A condição da cláusula HAVING é aplicada após a formação dos grupos; logo, podemos usar funções de agregação na construção das condições da cláusula HAVING.

Problema 25:

Listar os pedidos que possuam mais de um item.

Código:

```
SELECT no_ped,  COUNT(*) AS numero_itens

FROM item_pedido  GROUP BY no_ped  HAVING COUNT(*) > 1;
```

```
SELECT no_ped, COUNT(*) AS numero_itens FROM item_pedido GROUP BY no_ped HAVING COUNT(*) > 1;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela

Opções extras

no_ped	numero_itens
1111	3
1112	2
2111	2
2112	2
3112	2
4112	2
5111	2
6112	2

6.6 Recuperando Dados de Várias Tabelas (JOINS) 17/06/2026

Algumas consultas necessitam acessar simultaneamente várias tabelas, o que leva à realização de junções (JOINS) entre as tabelas para poder extrair as informações necessárias para a consulta formulada.

6.6.1 Qualificadores de Nomes

Um qualificador de nome consiste do nome da tabela, seguido de um ponto, seguido por um nome de uma coluna da tabela. Por exemplo, o qualificador da coluna DESC_PROD da tabela PRODUTO será PRODUTO.DESC_PROD. Os qualificadores de nome são utilizados em uma consulta para efetivar a junção (JOIN) entre as tabelas.

Problema 26 A:

Faça uma junção da tabela de clientes com a de pedidos, exibindo o nome do cliente, o código do cliente e o número do pedido.

Código:

SELECT cod_cli,nome_cli, pedido.num_ped

FROM cliente, pedido;

```
SELECT cod_cli, nome_cli, pedido.num_ped FROM cliente, pedido;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Pro

Opções extras

cod_cli	nome_cli	num_ped
1000	Supermercado Carrefour	1111
2000	Supermercado Baratao	1111
3000	Supermercado Arariboia	1111
4000	Mercado São João	1111
5000	CSN	1111
6000	Pegeout	1111
7000	Lojas Pague Pouco	1111
8000	Ford Caminhoes	1111
9000	Célula Celulose	1111
10000	Elevadores RioSul	1111
1000	Supermercado Carrefour	1112
2000	Supermercado Baratao	1112
3000	Supermercado Arariboia	1112

Neste exemplo foi executado um produto cartesiano das tabelas CLIENTE e PEDIDO, seguido de uma projeção das colunas exibidas. Neste caso, poucas informações úteis podem ser extraídas da relação resultante. Devemos então aplicar critérios de seleção à junção para podermos obter algum resultado concreto.

Problema 26 B:

A que clientes estão associados os pedidos existentes? Listar pelos nomes dos clientes.

Código:

select nome_cli, pedido.cd_cli, pedido.num_ped

from cliente, pedido

where cliente.cod_cli=pedido.cd_cli;

```
select nome_cli, pedido.cd_cli, pedido.num_ped from cliente, pedido where cliente.cod_cli = pedido.cd_cli;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela

Opções extras

nome_cli	cd_cli	num_ped
Supermercado Carrefour	1000	1111
Supermercado Carrefour	1000	1112
Supermercado Carrefour	1000	1113
Supermercado Arariboia	3000	2111
Supermercado Arariboia	3000	2112
Supermercado Arariboia	3000	2113
Mercado São João	4000	3111
Mercado São João	4000	3112
Pegeout	6000	4111
Pegeout	6000	4112
Ford Caminhoes	8000	5111
Célula Celulose	9000	6111
Célula Celulose	9000	6112
Elevadores RioSul	10000	7111

Questao 26 C

A equação apresentada na cláusula WHERE é chamada de EQUAÇÃO DE JUNÇÃO.

Podemos utilizar os operadores LIKE, NOT LIKE, IN, NOT IN, NULL, NOT NULL, os operadores relacionais e operadores AND, OR e NOT, na cláusula WHERE de uma junção de tabelas.

Problema 27:

Quais são os clientes que têm pedidos com prazos de entrega superiores a 15 dias e que estão localizados nos estados de São Paulo ou do Rio de Janeiro?

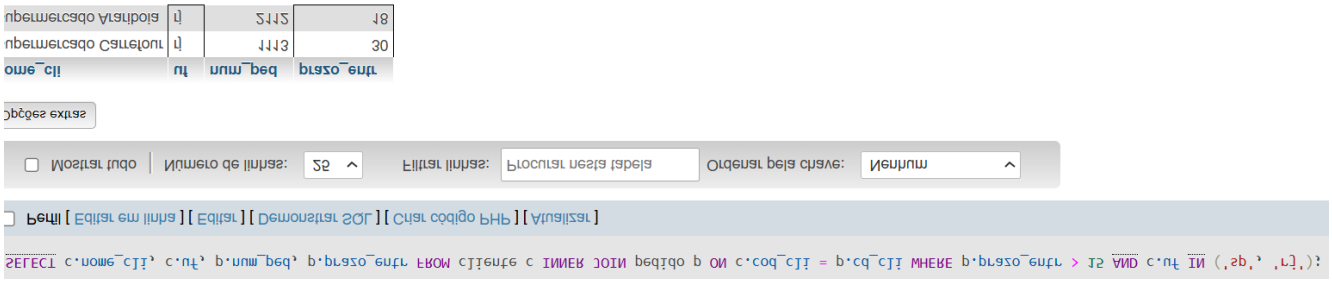
Código:

SELECT c.nome_cli, c.uf, p.num_ped, p.prazo_entr

FROM cliente c

INNER JOIN pedido p ON c.cod_cli = p.cd_cli

WHERE p.prazo_entr > 15 AND c.uf IN ('sp', 'rj');



Problema 28:

Mostrar os pedidos dos clientes e seus respectivos prazos de entrega, ordenados do maior para o menor.

Código:

```
SELECT c.nome_cli, p.num_ped, p.prazo_entr

FROM cliente c

INNER JOIN pedido p ON c.cod_cli = p.cd_cli

ORDER BY p.prazo_entr DESC;
```

```
SELECT c.nome_cli, p.num_ped, p.prazo_entr FROM cliente c INNER JOIN pedido p ON c.cod_cli = p.cd_cli ORDER BY p.prazo_entr DESC;
```

Perfil [Editar em linha] [Editar] [Demonstrar SQL] [Criar código PHP] [Atualizar]

Mostrar tudo | Número de linhas: 25 Filtrar linhas: Procurar nesta tabela Ordenar pela chave: Nenhum

Opções extras

nome_cli	num_ped	prazo_entr 1
Célula Celulose	6112	60
Supermercado Carrefour	1113	30
Célula Celulose	6111	30
Elevadores RioSul	7111	20
Supermercado Arariboia	2112	18
Supermercado Arariboia	2111	15
Mercado São João	3111	13
Ford Caminhoes	5111	10
Supermercado Carrefour	1111	10
Pegeout	4111	7
Pegeout	4112	7
Mercado São João	3112	7
Supermercado Carrefour	1112	5
Supermercado Arariboia	2113	3

Sinônimos 17/06/2026

Para que não seja necessário escrever o nome da tabela nas qualificações de nomes, podemos utilizar ALIASES definidos na própria consulta.

A definição dos ALIASES é feita na cláusula FROM, sendo então utilizada nas outras cláusulas (WHERE, ORDER BY, GROUP BY, HAVING e SELECT) de uma consulta.

Problema 29:

Exibir os vendedores (ordenados por nome) que emitiram pedidos com prazos de entrega superiores a 15 dias e que tenham salários fixos iguais ou superiores a R$ 1.000,00.

Código:

```
SELECT DISTINCT vendedor.nome_vend, vendedor.sal_fixo

FROM vendedor vendedor

INNER JOIN pedido produto ON vendedor.cod_vend = produto.cd_vend

WHERE produto.prazo_entr > 15

  AND vendedor.sal_fixo >= 1000.00

ORDER BY vendedor.nome_vend;
```